AI Tool Reflection — Buckeye Marketplace

AMIS 4630, Spring 2026

Overview

AI tools were used throughout every phase of this project — from initial architecture planning in Milestone 1 through production deployment in Milestone 6. The two primary tools were **GitHub Copilot** (inline code completion and agent mode in VS Code) and **Claude** (GitHub Copilot Chat, used for planning, debugging, and code generation). This document reflects on how those tools were used, what worked, what didn't, and what I learned about working effectively with AI in a real software development project.

How AI Was Used Across the SDLC

Milestone 1 — Requirements & Planning

AI (Claude via Copilot Chat) helped generate the initial project summary, feature list, and Kanban board structure. Prompts like "*I need to make a summary of my business system*" produced a solid starting point that I edited to match the actual scope. At this stage AI was most useful as a brainstorming accelerator — it surfaced ideas faster than starting from a blank page, but every output required review and editing to match the actual assignment requirements.

Milestone 2 — Architecture Design

AI helped draft the Architecture Decision Records (ADRs) and the component hierarchy using Atomic Design principles. The prompts were intentionally scoped: "*What technology should I use and why?*" and "*I need to use Atomic Design principles to create a component hierarchy. Scope to the Product Catalog feature.*" The outputs were useful starting points, but many of the initial technology choices (Node.js/Express, PostgreSQL, AWS) were placeholders that later changed as the project evolved — a good reminder that AI architectural advice needs validation against actual constraints.

Milestone 3 — Static Product Catalog

This milestone was the first time I used Copilot in **agent mode** to generate full-stack code. The agent created a React frontend (ProductListPage, ProductDetailPage, React Router) and a .NET API (GET /api/products, GET /api/products/{id}) from a single prompt. The generated code was reviewed file by file and accepted largely as-is. One correction was needed: I had to prompt separately to switch the .NET target framework to 10.0.3 and to add the exact product fields required by the rubric.

Milestone 4 — Cart, Persistence & Integration

The most iterative milestone. AI was used in six steps, with each step building on the previous. Copilot generated CartContext with useReducer, CartController with EF Core SQLite persistence, stock enforcement, quantity pickers, and a full service/hook refactor. The generated code quality was high, but several modifications were required:

- **CartContext revision:** The initial generation switched state management from `useReducer` to `useState`. I prompted for a correction to preserve `useReducer` as required, which Copilot correctly applied.
- **Stock enforcement:** Required a separate follow-up prompt after the initial cart implementation — the original prompt didn't specify it.

This milestone demonstrated a key pattern: AI works best with narrow, well-specified prompts. Broad prompts produced code that required more corrections than targeted ones.

Milestone 5 — Authentication, Orders & Testing

AI was used in a more structured way, with purpose-built agents for testing and security review:

Testing agent: Used `TESTING-AGENT.md` to guide Copilot through a two-phase process: first planning what to test (returning a table of candidates with rationale), then generating the tests. This separation of planning from execution produced better tests than asking for tests directly.

Two bugs caught in generated tests:

1. `ALLUPPERCASE1` was expected to fail validation — Copilot assumed the Identity policy required a lowercase letter, but reading `Program.cs` confirmed it didn't. Fixed manually.
2. The `CustomWebApplicationFactory` created a new SQLite in-memory connection per DbContext registration, so the schema disappeared between requests. This was a subtle bug that took debugging to find — Copilot didn't catch it initially.

Security audit agent: Used a structured prompt to audit JWT configuration, CORS, SQL injection risk, and authorization. Found a genuine issue (JWT expiration set to 8 hours) that I then fixed.

Milestone 6 — Production Deployment & CI/CD

This milestone involved the most back-and-forth debugging with AI, primarily because deployment errors are environment-specific and hard for AI to anticipate without live feedback. The major issues resolved with AI assistance:

1. **Zip deploy — a prolonged and frustrating dead end.** This was the most time-consuming issue of the entire project, and AI was a direct contributor to the delay. The initial GitHub Actions workflow generated by AI used `dotnet publish` followed by `Compress-Archive` to create a `.zip` file, then deployed it via the Kudu REST API (`/api/zipdeploy`). This approach failed with rsync exit code 23 and then a series of 401 Unauthorized errors. AI's successive suggestions — changing the zip path, switching to `$` SCM credentials, base64-encoding the Authorization header — each appeared reasonable in isolation but none resolved the underlying problem. After multiple failed CI runs and significant time lost, the actual fix was to abandon zip deploy entirely and use `azure/webapps-deploy@v3` with the publish folder directly, which is how the official Microsoft sample apps work. The lesson: AI confidently proposes plausible-sounding solutions even when it doesn't have enough context to know whether they'll work. When multiple AI-suggested fixes fail in sequence, it's a signal to stop and find a working reference implementation rather than continuing to iterate on the same broken approach.
2. **Frontend blank white page** — caused by incorrect `app_location` / `output_location` in the SWA workflow.
3. **Azure container startup timeout** — caused by synchronous DB initialization blocking the HTTP health check. Fixed by moving all DB init to a `BackgroundService` (`DbInitializerService`), which was a pattern AI suggested after explaining the Azure 230-second warmup constraint.
4. **SQL login failure** — the wrong password was stored in the Azure App Service connection string. Diagnosed from the container logs and fixed by updating the connection string via Azure CLI.
5. **CORS missing Allow-Origin header** — fixed by adding `appsettings.Production.json` with the production frontend origin, which ASP.NET Core automatically loads when `ASPNETCORE_ENVIRONMENT=Production` .

What Worked Well

Agent mode for boilerplate-heavy code. Features like the full cart implementation (Context, reducer, API layer, components, backend controller, EF migrations) would have taken hours manually. AI completed the scaffolding in minutes, leaving time to focus on reviewing correctness and edge cases.

Structured prompting for tests. The two-phase approach (plan first, generate second) consistently produced better tests than asking for tests directly. Asking AI to explain *why* a test matters before writing it filtered out tests that would have just duplicated the implementation.

Debugging with AI as a thought partner. For hard-to-diagnose issues (like the Azure startup timeout), explaining the problem to AI and asking it to identify possible root causes was faster than reading documentation alone. Even when AI's first suggestion wasn't right, the process of ruling it out clarified the actual problem.

Code review by prompting AI to audit its own output. Asking "*audit this for security issues before we move on*" after each feature caught real problems (JWT expiration, missing role checks) that might otherwise have been missed.

What Didn't Work Well

AI cannot see live environment state. Every deployment issue required copy-pasting logs from Azure into the chat. AI gave good analysis once it had the logs, but it couldn't proactively detect that the SQL password was wrong or that the container was timing out — it could only react to evidence I provided.

Confident suggestions are not always correct suggestions — especially for deployment. The most frustrating experience of the project was the zip deploy loop. AI generated a GitHub Actions workflow that zipped the publish output and deployed it via the Kudu REST API. When it failed with rsync exit code 23, AI suggested changing paths. When that produced 401 errors, AI suggested switching credential formats, then base64-encoding the Authorization header. Each suggestion was presented confidently and sounded plausible, but none worked — and following them cost multiple CI run cycles and significant time. The real fix was to find the official Microsoft sample app and replicate its pattern (`azure/webapps-deploy@v3` with the publish folder), which had nothing to do with any of AI's suggestions. The pattern to watch for: if you are on your third or fourth AI-suggested fix for the same issue and none have worked, stop iterating and find a canonical reference instead.

First-generation code often needs correction. Roughly 30% of generated code required at least one follow-up prompt or manual edit. Common issues: wrong assumptions about which packages were installed, generating code that worked for a simpler version of the requirement, or missing edge cases like the SQLite in-memory test factory connection lifetime.

Architecture decisions degrade without human judgment. Early AI-suggested technology choices (Node.js, PostgreSQL, AWS) were replaced by the time development started. AI optimizes for common patterns, not for specific constraints like course requirements, existing team knowledge, or Azure free-tier availability. These decisions needed human judgment.

Long sessions lose context. In extended debugging sessions, AI occasionally contradicted earlier advice or forgot about a fix that had already been applied. Keeping prompts self-contained and re-stating relevant context at the start of each question helped, but wasn't a perfect solution.

Impact on Productivity and Learning

AI tools meaningfully accelerated development. A realistic estimate is that features took 40–60% less calendar time to implement than they would have without AI assistance. The biggest time savings came in:

- Initial scaffolding of new features (cart, auth, orders)
- Writing boilerplate (EF migrations, JWT configuration, React Context wiring)

- Diagnosing deployment errors from log output

However, time savings came with a trade-off: it was possible to accept generated code without fully understanding it. On several occasions, debugging required going back and reading code carefully that had been accepted too quickly. The most valuable learning happened not from watching AI generate code, but from debugging the cases where it was wrong — those forced a deep understanding of how the JWT authentication flow worked, how EF Core in-memory SQLite connections behave, and how Azure App Service initializes containers.

Lessons Learned

1. **Review everything, merge nothing blindly.** Every file AI generated was reviewed before being committed. The two integration test bugs described above — and several smaller issues — were only caught because of careful review.

2. **Narrow prompts outperform broad ones.** "*Add stock enforcement to the cart POST endpoint*" produced better output than "*improve the cart.*" Specificity is the most important factor in prompt quality.

3. **Use AI for the what, not the why.** AI is fast at generating implementations. Understanding *why* a particular approach is correct — why a `BackgroundService` solves the Azure startup timeout, why JWT claims must come from the token and not the request body — requires reading documentation and thinking it through independently.

4. **Treat AI suggestions as pull requests, not commits.** The mental model of reviewing AI output the way you'd review a colleague's PR produces better outcomes than either rubber-stamping everything or ignoring AI suggestions entirely.

5. **Document AI use as you go.** The `AI-USAGE.md` log maintained throughout the project made writing this reflection far easier and more accurate than reconstructing it from memory would have been.

Tools Used

Tool	How Used
GitHub Copilot (inline)	Code completion, single-function generation, quick edits
GitHub Copilot Chat / Claude	Multi-file feature generation, debugging, architecture questions, security audits
Copilot Agent Mode	Full-feature scaffolding (cart, auth, tests, CI/CD workflows)
Custom `.prompt.md` agents	Structured testing agent, security audit agent

---

Total AI-assisted development sessions: ~15 across 6 milestones Estimated AI-generated code accepted without modification: ~65% Estimated AI-generated code requiring correction or follow-up: ~35%

Buckeye Marketplace — AMIS 4630, Spring 2026